Lecture 18: Classes (Chapter 6)

유준수 (Joon Soo Yoo, Ph.D.)

2026년 4월 14일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

11.4 Overloading Operators

매직 메서드(Magic Methods)를 이용한 객체 연산 커스텀

매직 메서드의 필요성: 왜 사용할까?

우리가 정의한 클래스 객체가 파이썬의 기본 자료형(int, list 등)처럼 **자연스럽게** 동작하게 하려는 목적입니다.

1. 직관적인 구문 (Syntactic Sugar)

메서드 이름을 일일이 외울 필요 없이 표준 연산자(+, -, *)를 그대로 사용할 수 있습니다.

- **Before:** `result = account1.add_balance(account2)`
- **After:** `result = account1 + account2`

2. 파이썬 내장 기능과의 완벽한 통합

`print()`, `len()`, `str()` 같은 파이썬 기본 함수들이 우리가 만든 객체에서도 즉시 동작하도록 인터페이스를 표준화합니다.

팁

좋은 코드는 읽기 쉬워야 합니다. `a + b`라고 쓰면 누가 봐도 더하기인 줄 알지만, `a.complex_internal_add(b)`라고 쓰면 코드를 한참 분석해야 합니다.

매직 메서드(Magic Methods)란?

이름 앞뒤에 밑줄 두 개(__)가 붙어 '더블 언더스코어' 또는 **던더(Dunder)** 메서드라고도 불립니다.

주요 특징

- 사용자가 직접 호출하지 않아도 파이썬 시스템에 의해 자동으로 실행됩니다.
- `__init__()`: 객체 초기화 시 자동 실행
- `__str__()`: `print()` 함수가 객체를 문자열로 변환할 때 자동 실행

왜 '매직'인가요?

연산자(+)를 썼는데 내부적으로 메서드(`__add__`)가 실행되는 모습이 마치 마법 같다고 해서 붙여진 이름입니다.

객체의 명함: __str__() 메서드

객체를 print()하거나 str()로 변환할 때 호출되는 매직 메서드입니다.

실전 예제: 계좌 정보 출력

```
class Account:
  def __init__(self, name, balance):
    self.name = name
    self.balance = balance

  def __str__(self):
    return f"계좌[{self.name}]: 잔액 {self.balance}원"

my_acc = Account("민석", 100000)
print(my_acc)   # 결과: 계좌[민석]: 잔액 100000원
```

중요 규칙

__str__ 메서드는 반드시 **문자열(String)** 타입을 반환(return)해야 합니다. 숫자를 반환하면 에러가 납니다!

산술 연산자 오버로딩

우리가 만든 클래스의 인스턴스끼리 더하거나 뺄 때 어떤 동작을 할지 정의합니다.

주요 산술 매직 메서드

- `a + b` → `a.__add__(b)`
- `a - b` → `a.__sub__(b)`
- `a * b` → `a.__mul__(b)`

예시: 계좌 잔액 더하기

```
def __add__(self, other):
    return Account(self.name + " & " + other.name, self.amount + other.amount)
```

비교 연산자 오버로딩

객체 간의 크기를 비교(<, >, ==)하는 기준을 세웁니다. 결과는 항상 **Boolean(True/False)**을 반환해야 합니다.

비교 매직 메서드 매핑

- `a < b` → `__lt__(self, other)` (Less Than)
- `a > b` → `__gt__(self, other)` (Greater Than)
- `a == b` → `__eq__(self, other)` (Equal)

데이터 기반 비교

단순히 객체 자체가 같은지 보는 게 아니라, 객체 내부의 `amount`나 `id` 같은 특정 속성값을 기준으로 비교하게 만듭니다.

Task 1: 계좌 통합 관리

연구실의 각 프로젝트 계좌를 합치거나 예산을 지출하는 시스템을 설계해 봅시다.

Mission: Account 클래스에 산술 연산 기능 추가하기

1. **초기화:** `name`(계좌명)과 `balance`(잔액)를 속성으로 가집니다.
2. **더하기 오버로딩 (__add__):** 두 계좌를 더하면 잔액은 합산되고, 이름은 "A & B" 형태로 결합된 **새로운 Account 객체**를 반환하세요.
3. **빼기 오버로딩 (__sub__):** 계좌에서 숫자(정수)를 빼면 잔액이 그만큼 차감되도록 구현하세요. (예: `acct - 5000`)
4. **출력 설정 (__str__):** `print()` 호출 시 "계좌[이름]: 잔액원" 형식으로 출력되게 하세요.

실행 결과 예시

```
a1 = Account("민석 과제", 100000)
a2 = Account("용준 과제", 200000)
a3 = a1 + a2        # 계좌[민석 과제 & 용준 과제]: 300000원
a3 = a3 - 50000    # 계좌[민석 과제 & 용준 과제]: 250000원
```

11.5 Using Modules with Classes

클래스와 모듈의 결합: 코드의 재사용성과 조직화

왜 클래스를 모듈로 관리할까?

클래스가 많아지면 하나의 파일에 모든 코드를 담기 어렵습니다. 관련 있는 클래스들을 모듈(.py 파일)로 그룹화하면 관리가 쉬워집니다.

모듈화의 장점

- **재사용성(Reusability):** 잘 만들어둔 클래스를 다른 프로젝트에서도 그대로 `import`해서 쓸 수 있습니다.
- **가독성:** 기능별로 파일이 나뉘어 있어 대규모 시스템을 파악하기 좋습니다.
- **협업:** **A**는 센서 클래스를, **B**는 제어 클래스를 각자 파일에서 개발할 수 있습니다.

클래스를 불러오는 두 가지 방법

어떤 방식으로 임포트하느냐에 따라 클래스 호출 방식이 달라집니다.

방법 A: 모듈 전체 임포트

```
import character

hero = character.Level()
# 모듈 이름을 앞에 붙여야 함
```

방법 B: 특정 클래스만 임포트

```
from character import Level

hero = Level()
# 클래스 이름을 바로 사용 가능
```

팁

방법 B가 코드는 더 짧지만, 다른 곳에서 정의한 이름과 겹칠 수 있으니 주의해야 합니다!

별칭(Alias) 사용하기: `import ... as`

모듈 이름이 너무 길거나, 이름 충돌(Name Collision)이 걱정될 때 짧은 별명을 붙여 사용합니다.

별칭 활용 예시

```
import defense_technology_module as dtm


sensor = dtm.LidarSensor()
# 길고 복잡한 이름을 dtm으로 줄여서 사용
```

주의사항

별칭을 너무 무분별하게 쓰면 다른 개발자가 코드를 읽을 때 혼란스러울 수 있습니다. 관례적으로 많이 쓰는 별칭(예: `pd`, `np`)이 아니라면 신중하게 정해야 합니다.

Task 2: 나만의 게임 캐릭터 불러오기

여러 클래스를 별도의 파일(모듈)로 나누어 관리하고, 게임 서버에서 이를 불러오는 연습을 해봅시다.

Mission: 캐릭터 모듈 생성 및 임포트

1. **파일 1 (game_chars.py):** `Warrior` 클래스와 `Mage` 클래스를 정의하세요.
2. **파일 2 (main.py):** `from game_chars import Warrior`를 사용하여 전사만 가져와 인스턴스를 만드세요.
3. **별칭 연습:** `import game_chars as gc`를 사용하여 `gc.Mage()`로 마법사 인스턴스를 만드세요.
4. **검증:** 각 캐릭터의 공격(`attack`) 메서드를 호출하여 정상적으로 출력되는지 확인합니다.

팁

리그 오브 레전드 같은 큰 게임도 수백 명의 챔피언 정보를 이런 식으로 모듈화해서 관리합니다. 코드가 깔끔해지는 마법입니다.

13.1 Inheritance Basics

코드의 가업 잇기: 상속의 개념과 구조

상속(Inheritance)이란?

이미 만들어진 클래스(부모)의 모든 속성과 메서드를 새로운 클래스(자식)가 그대로 물려받는 것입니다.

- **중복 제거:** 똑같은 코드를 여러 번 쓸 필요가 없습니다.
- **확장성:** 부모의 기능을 기반으로 새로운 기능만 추가하면 됩니다.
- **유지보수:** 부모 클래스만 잘 수정하면 이를 상속받은 모든 자식 클래스에 반영됩니다.

설계의 핵심: is-a vs has-a

클래스 간의 관계를 정확히 정의해야 올바른 상속을 할 수 있습니다.

is-a (상속 관계)

"자식은 부모의 한 종류이다."
전사(`Warrior`) **is-a** 캐릭터(`Character`)
→ 상속을 사용합니다.

has-a (포함 관계)

"객체가 다른 객체를 가지고 있다."
전사 **has-a** 칼(`Sword`)
→ 상속이 아닌 '변수'로 포함합니다.

상속 vs 포함: 헷갈릴 때 던지는 질문

1. 상속(is-a): "A는 B이다" (분류학적 관계)

하위 클래스가 상위 클래스의 **개념적 범주** 안에 완전히 포함될 때 사용합니다.

- **자연스러운 표현:** "참새는 새다", "노트북은 컴퓨터다"

2. 포함(has-a): "A는 B를 가지고 있다" (소유/구성 관계)

한 객체가 다른 객체를 **부품이나 도구**로 사용할 때 사용합니다.

- **자연스러운 표현:** "자동차는 엔진을 가지고 있다", "학생은 가방을 메고 있다"

설계 가이드

전사 `is a` 칼? (X) → 전사가 칼일 수는 없으니 상속 금지!
전사 `has a` 칼? (O) → 전사 클래스 안에 칼 변수를 만드세요.

파이썬 상속 문법

클래스를 정의할 때 괄호 안에 부모 클래스의 이름을 적어줍니다.

상속 기본 구조

```
class Parent:   # 부모 클래스 (Superclass)
    def hello(self): print("안녕하세요")

class Child(Parent):   # 자식 클래스 (Subclass)
    pass

c = Child()
c.hello()   # 부모의 메서드를 내 것처럼 사용!
```

함수(Function) vs 메서드(Method)

둘 다 특정 기능을 수행하는 코드 블록이지만, "누구에게 속해 있는가"가 핵심입니다.

함수 (Function)

독립적으로 존재하며 이름을 통해 직접 호출합니다.

- 예: `print()`, `len()`, `sum()`
- 특정 객체의 상태를 직접 참조하지 않습니다.

```
result = sum([1, 2, 3])
```

메서드 (Method)

클래스 내부에 정의되어 객체를 통해 호출합니다.

- 예: `my_list.append()`, `a1.attack()`
- `self`를 통해 객체의 데이터(속성)에 접근합니다.

```
a1.attack(a2)
```

결론: 메서드는 "객체가 수행하는 함수"라고 이해하면 된단다!

누가 무엇을 사용할 수 있나?

상속은 일방통행입니다. 자식은 부모의 것을 다 알지만, 부모는 자식의 전용 기능을 모릅니다.

접근 권한 정리

- **자식 인스턴스:** 부모 메서드 + 자식 메서드 **모두** 사용 가능
- **부모 인스턴스:** 부모 메서드만 사용 가능 (자식 기능 접근 불가)

용어	부모 클래스	자식 클래스
일반 용어	Superclass (슈퍼 클래스)	Subclass (서브 클래스)
기초 용어	Base class (기초 클래스)	Derived class (파생 클래스)
가족 비유	Parent class (부모 클래스)	Child class (자식 클래스)

Task 3: RPG 캐릭터 전직 시스템 설계

캐릭터가 전직할 때마다 이전 단계의 기술을 그대로 간직하며 새로운 기술을 배우는 '상속의 흐름'을 구현해 봅시다.

Mission: 3단계 전직 구조 만들기

1. **GameCharacter (초보자):** `move()` 메서드 ("기본 이동을 합니다")
2. **Warrior (전사):** `GameCharacter`를 상속받고, `slash()` 메서드 ("칼을 휘두릅니다")
3. **Berserker (광전사):** `Warrior`를 상속받고, `rage()` 메서드 ("폭주 상태가 됩니다!")
4. **실험:** `Berserker` 인스턴스를 하나 만들고, 초보자 기술(`move`), 전사 기술(`slash`), 광전사 기술(`rage`)을 모두 호출해 보세요.